CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Experiment: 8

Name: Piyush Patel
UID: 23BCS12673
Branch: BE-CSE
Section: KRG-2A
Semester: 6th
Date of performance:08/04/2026
Subject: System Design
Subject Code: 23CSH-314

1. AIM

To design a stock-trading platform (Similar to Zerodha, Grow & Upstock). These platforms serves as the commission-free online platform for trading and monitoring of the stocks.

2. OBJECTIVE:

- Enable secure user onboarding and authentication with KYC/AML compliance integrated into the system design.
- Ingest and process real-time market data feeds from exchanges with scalable, fault-tolerant streaming architecture.
- Design a trading engine microservice to handle order placement, modification, cancellation, and exchange integration.
- Implement portfolio and settlement services to track holdings, P&L, and clearing house interactions.
- Ensure scalability, reliability, and compliance through event-driven architecture, audit logging, and high-availability deployment.

3. FUNCTIONAL REQUIREMENTS

- Users should be able to do the registration, login and KYC verification.
- Application should allow users to view real time stocks prices & its historical data.
- Application should support placing, modifying, and cancelling orders along with limiting orders with accurate status updates.
- Application should provide users with a watchlist with real time updates.
- Application should provie a dashboard to track current holdings, trade history, P& L, and performance insights.

Core-Entities of the System:

-Users -Trade
-Portfolio -Watchlist
-Stocks -Orders

4. NON-FUNCTIONAL REQUIREMENTS

- Scale: (2 - 3M)+ DAU with high-frequency trades, along with 8 - 10k stocks.
- CAP Theorem: Consistency >>> Availability
    - Correctness of stocks is more important than uptime. (A wrong balance, duplicate trade, or stale price can lead to huge financial losses)
    - Getting the stocks prices in real-time (highliy available)

- Latency: Order placement < 100ms, market data (updated stock value) < 50ms

5. API DESIGN

Users:

1. POST: / auth / signup
2. POST: / auth / verify-KYC
3. POST: / auth / login

Market Data:

1. [Listing of all the stocks] WS: / api / v1 / stocks
2. [stock details] GET: / api / v1 / stocks / (stock_symbol_id)
3. [Historical Data] GET: / api/ v1 / stocks / {stock_symbol_id} / history

Funds:

1. POST: / api / v1 / funds / deposit
2. POST: / api / v1 / funds / withdraw
3. GET: / api / v1 / funds / history

Trading:

1. [Place buy / sell order] POST: / api / v1 / orders
2. [List user orders] GET: / api / v1 / {user_id} / orders
3. [Order Status] GET: / api / v1 / orders / {order_id}
4. [Cancel order] DELETE: / api / v1 / orders / [order_id]

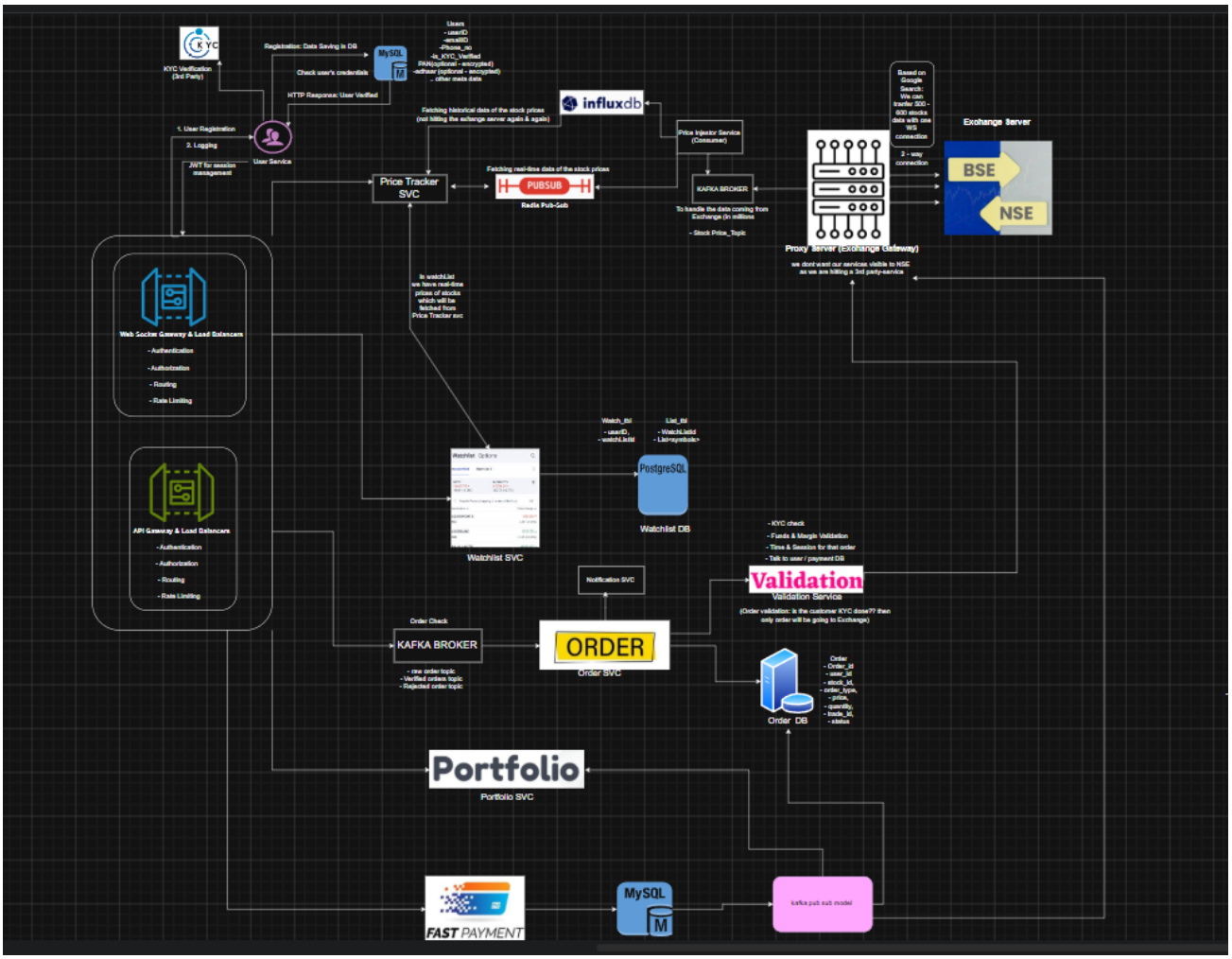
Portfolio:

1. GET: / api / v1 / portfolio
2. GET: / api / v1 / portfolio / holdings
3. GET: / api / v1 / portfolio / positions
4. GET: / api . v1 / portfolio / performance [OPTIONAL]

Watchlist:

1. GET: / api / v1 / watchlists
2. POST / api / v1 / watchlists / {watchlist_id} / symbols

6. HIGH-LEVEL DESIGN



7. LEARNING OUTCOMES

- Understood the architecture of real-world trading systems and how they integrate with exchanges and clearing houses.
- Identified and modeled core entities such as User, Account, Order, Trade, Portfolio, Market Data, and Settlement.
- Learnt how to design scalable APIs for trading operations, portfolio management and real-time market data queries.
- Explored risk management and compliance mechanisms including margin checks, exposure limits, and audit logging.
- Developed insights into distributed system constraints like consistency, fault tolerance, and high availability in financial platforms.